Факультет компьютерных технологий и прикладной математики

Кафедра анализа данных и искусственного интеллекта

09.03.03

Современные среды разработки интерактивных приложений

Лабораторная работа № 1

Первый прототип журнала заездов на Windows Forms

1. Цель работы

Создать собственное приложение Windows Forms с нуля. Научиться работать с конструктором формы, задавать свойства элементов, связывать нажатия кнопок с кодом, проверять введённые данные и показывать результат в интерфейсе. Работа рассчитана на два занятия по два академических часа.

Вы создадите журнал тренировочных заездов. Пользователь введёт пилота и время круга, выберет автомобиль и трассу, добавит результат в список. Отдельные кнопки позволят удалить выбранную запись и очистить поля нового ввода. Список будет храниться только до закрытия приложения.

В начале работы готового проекта нет. Проект, элементы формы и обработчики создайте самостоятельно в Microsoft Visual Studio. Маленький пример ниже показывает только добавление имени. На его основе вам предстоит написать остальное поведение журнала.

2. Что понадобится

Работу выполняют в Windows с Microsoft Visual Studio 2026. В установленной среде нужна рабочая нагрузка «Разработка классических приложений .NET» (.NET desktop development) и компонент .NET 10. Проект создаётся для .NET 10 LTS. Visual Studio Code для описанного порядка действий не используется.

Названия команд приведены на русском языке. При первом упоминании рядом указаны английские названия. На снимках показан английский интерфейс и проект Names. В этой работе назовите свой проект RaceLog.

Расположение панелей и тема оформления могут отличаться, поэтому находите окна по названию.

Как выполнять работу

На первом занятии пройдите этапы 1-5: создайте проект и получите работающий пример с именем. На втором выполните этапы 6-9: расширьте интерфейс, напишите самостоятельную часть и проверьте её. После каждого этапа указан результат, с которым нужно свериться. Не переходите дальше, пока последняя рабочая версия не запускается.

3. Необходимая теория: приложение и его части

Среда разработки объединяет инструменты для создания интерфейса, ввода кода, сборки и отладки. Visual Studio не является языком программирования. Поведение приложения описывается на C#, а .NET предоставляет необходимые библиотеки и среду выполнения. Windows Forms предоставляет окна и элементы управления для Windows.

Проект, решение и форма

Проект содержит код приложения и его настройки. Решение объединяет один или несколько проектов. В ЛР 1 достаточно одного проекта в одном решении. Форма является окном приложения. В конструкторе формы вы размещаете элементы, а в редакторе кода описываете их поведение. Это два способа работы с одной формой, а не два разных приложения.

Класс и объект

Класс описывает устройство и поведение объектов. Шаблон создаёт класс Form1, который описывает нашу форму. При запуске создаётся объект этого класса, то есть конкретное окно. Готовый класс Form предоставляет обычные возможности окна. Form1 использует их и дополняется вашим кодом. Создавать классы «Пилот» или «Заезд» в этой работе пока не требуется.

Свойство, метод, событие

Свойство позволяет прочитать или изменить значение, например видимый текст кнопки. Метод выполняет действие, например очищает поле.

Событие сообщает, что что-то произошло, например пользователь нажал кнопку. Обработчик события является методом, который программа вызывает в ответ. У кнопки событие называется Click, а метод нашего приложения будет называться btnAdd_Click.

Понятие	**Пример**	**Как читать**
Свойство	txtDriver.Text	Текст поля имени
Метод	txtDriver.Clear()	Очистить поле имени
Событие	Click	Пользователь нажал кнопку
Обработчик	btnAdd_Click	Метод реакции на нажатие

Пока не набирайте эти примеры в редакторе. Сначала создайте форму и элементы на этапах 1-4. Таблица нужна, чтобы узнавать назначение терминов в следующих шагах.

Этап 1. Открытие среды и проверка компонентов

1.1. Запустите среду

Откройте меню «Пуск» Windows, введите Visual Studio 2026 и запустите найденное приложение. Не выбирайте Visual Studio Code или Visual Studio Installer вместо основной среды. На начальном экране найдите «Создать проект» (Create a new project). Если Visual Studio уже открыта с другой работой, используйте «Файл», «Создать», «Проект» (File, New, Project).

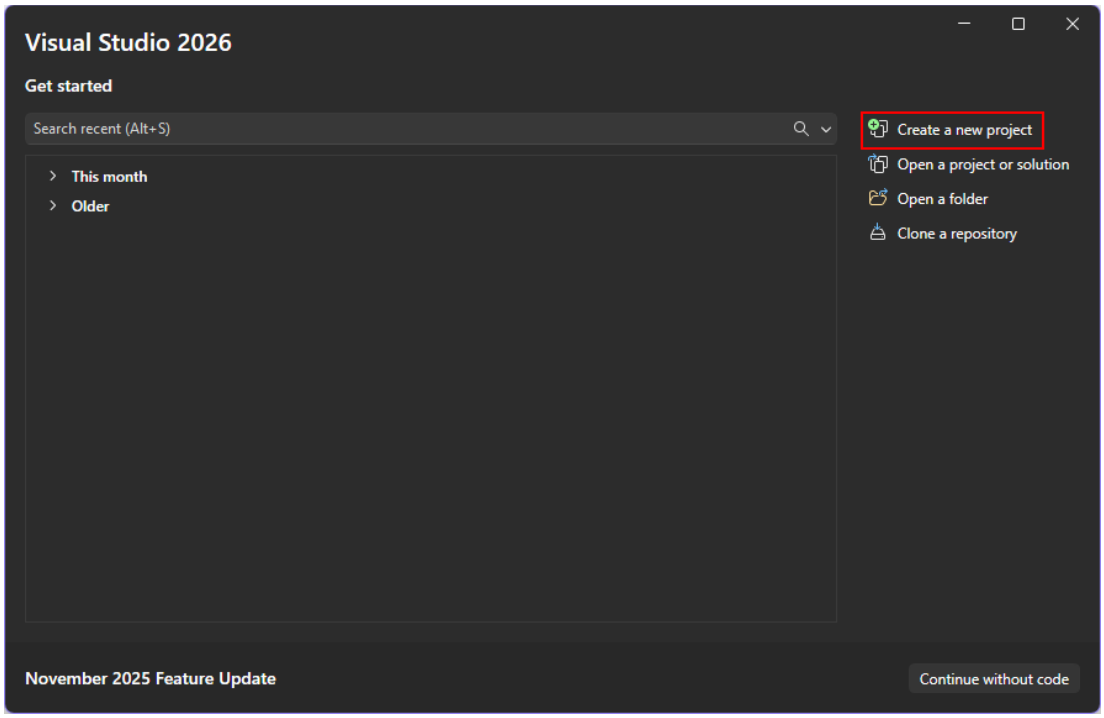


Рисунок 1 – Начальное окно Visual Studio 2026, выберите Create a new project

1.2. Проверьте наличие шаблона

В поиске шаблонов введите winforms. Если результаты отсутствуют, сначала сбросьте лишние фильтры. Затем проверьте установленные компоненты. Для этого откройте Visual Studio Installer через «Пуск», найдите установленную Visual Studio 2026 и выберите «Изменить» (Modify). На вкладке рабочих нагрузок должна быть отмечена .NET desktop development.

В списке отдельных компонентов проверьте наличие .NET 10. После изменения установки дождитесь завершения и снова откройте Visual Studio. Не устанавливайте все нагрузки подряд. Для ЛР 1 не требуются мобильная разработка, Unity, C++ и внешние библиотеки. Если у вас нет прав на изменение учебного компьютера, сообщите преподавателю, какого компонента не хватает.

Что должно получиться

Visual Studio запущена, окно создания проекта открывается, поиск winforms показывает нужный шаблон. Пока не требуется ни скачивать чужое решение, ни создавать вручную отдельные заготовки кода.

Если не получается

Если при поиске находится только Visual Studio Code, основная среда не установлена или не добавлена в меню. Если шаблон виден, но нет нужной платформы, это отдельная проблема компонента .NET 10. Не заменяйте выбранную платформу на .NET Framework, чтобы обойти отсутствие компонента.

Этап 2. Создание своего проекта

2.1. Выберите шаблон

В окне «Создание проекта» укажите C# в фильтре языка. Выберите Windows Forms App без приписки .NET Framework. В русской локализации название может быть переведено, поэтому дополнительно смотрите метку языка C# и описание шаблона. Нажмите «Далее» (Next).

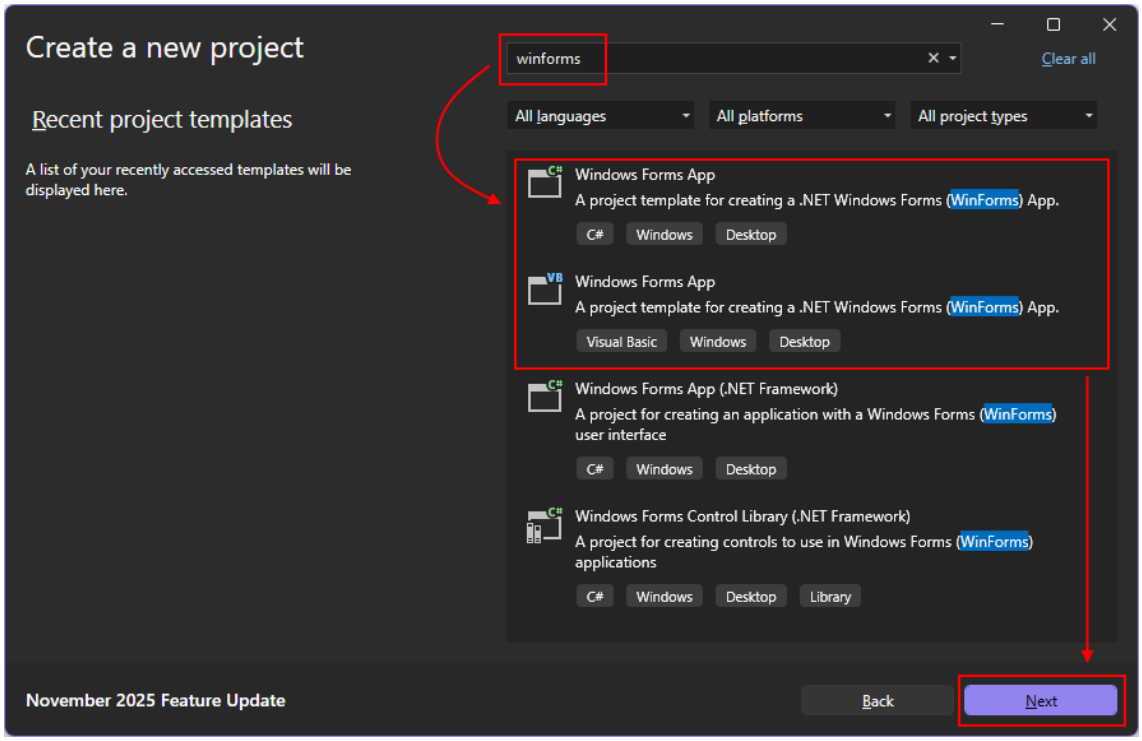


Рисунок 2 – Выбор шаблона Windows Forms App для C#, на снимке показаны два языка, в работе нужен шаблон с меткой C#

2.2. Задайте имя и расположение

В поле Project name введите RaceLog. Это внутреннее имя приложения. В поле Location выберите постоянную рабочую папку, где у вас есть право записи. Например, создайте в своих документах папку «ССРИП», а внутри неё

папку «ЛР1». Запомните выбранное место, чтобы потом открыть и передать всю работу. Название решения можно оставить RaceLog.

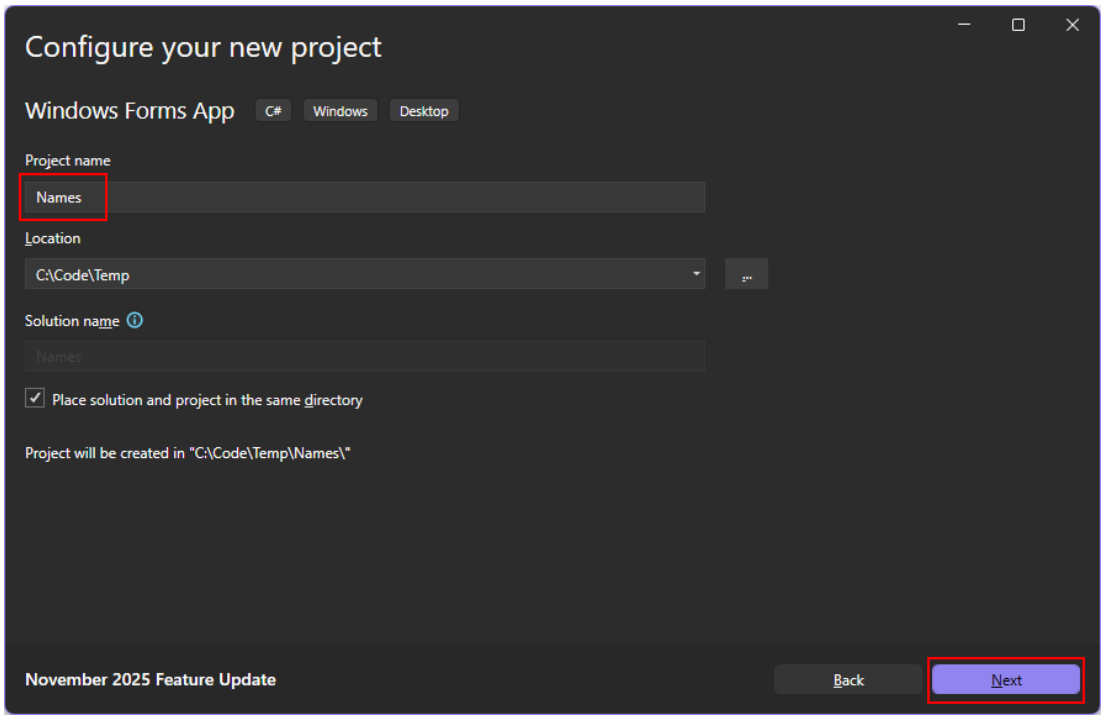


Рисунок 3 – Имя и расположение проекта в примере, на снимке проект назван Names, в нашей работе введите RaceLog и выберите свою рабочую папку

Чужой путь расположения не копируйте. Имя проекта на латинице удобно использовать в программных именах, а русский заголовок окна задаётся позже отдельным свойством.

2.3. Выберите платформу

В следующем окне Additional information установите Framework: .NET 10.0 (Long Term Support). Нажмите «Создать» (Create) и дождитесь завершения подготовки. Не выбирайте предварительную версию платформы.

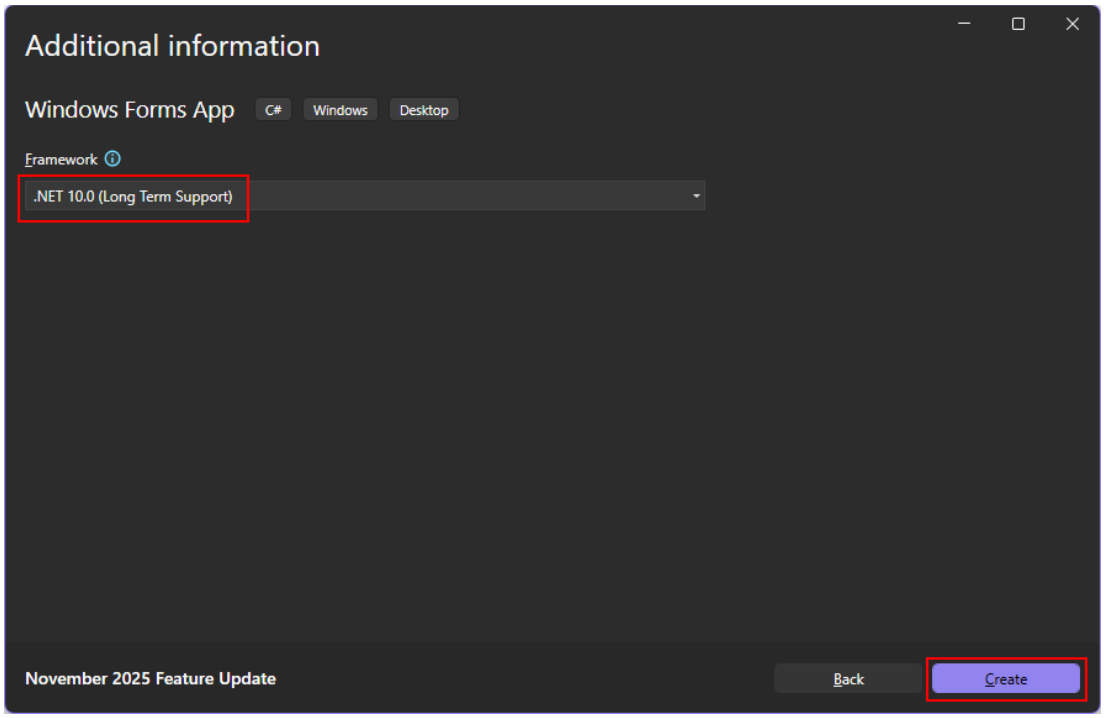


Рисунок 4 – Выбор .NET 10 LTS, в Framework выберите .NET 10.0 (Long Term Support)

Что должно получиться

Открылось решение RaceLog с одним проектом. В центре видна пустая форма Form1. Это форма, которую создал стандартный шаблон в вашем новом проекте. Готовой формы журнала заездов в ней ещё нет. Имя Form1 пока не изменяйте.

Этап 3. Окна среды, расположение работы и первый запуск

3.1. Найдите основные окна

Откройте «Вид» (View). Выберите «Обозреватель решений» (Solution Explorer), «Панель элементов» (Toolbox) и «Окно свойств» (Properties Window). Названия комбинаций клавиш зависят от выбранного профиля, поэтому основной путь здесь указан через меню.

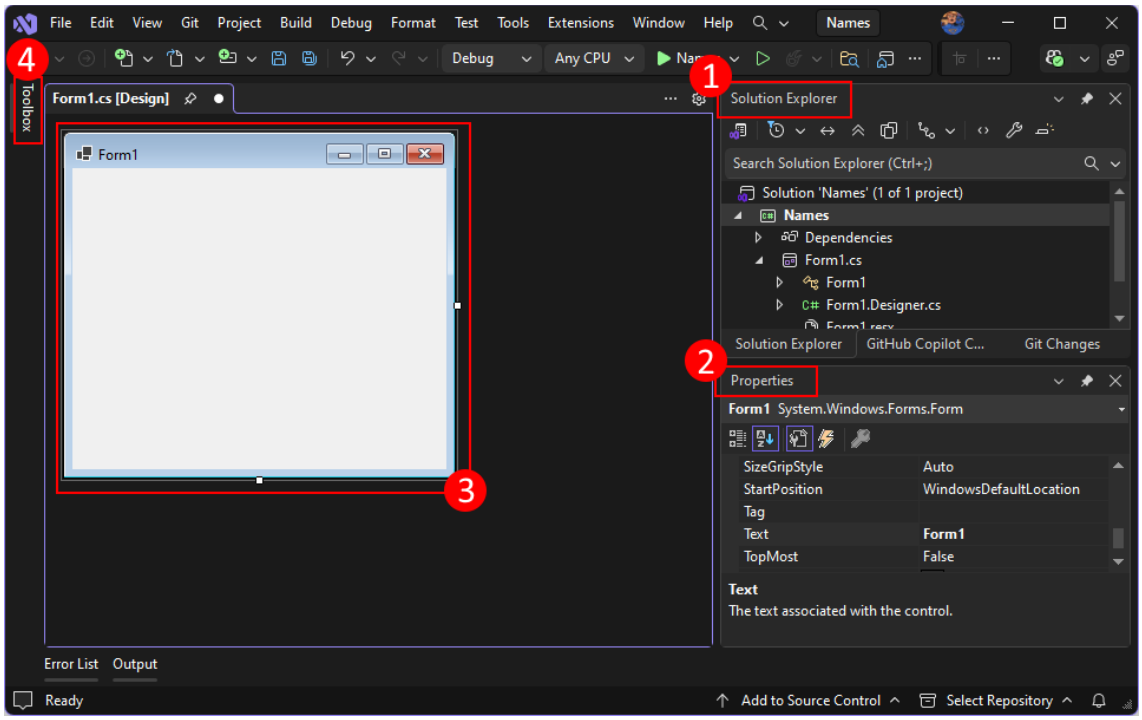


Рисунок 5 – Основные области Visual Studio, 1: Solution Explorer. 2: Properties. 3: Form Designer. 4: Toolbox

Solution Explorer показывает состав проекта. Toolbox содержит элементы, которые перетаскивают на форму. Properties показывает параметры выбранного объекта. Form Designer является центральной поверхностью для размещения элементов. Если Toolbox пуст, сначала откройте конструктор Form1. Для этого в Solution Explorer щёлкните Form1 правой кнопкой и выберите «Открыть конструктор» (View Designer).

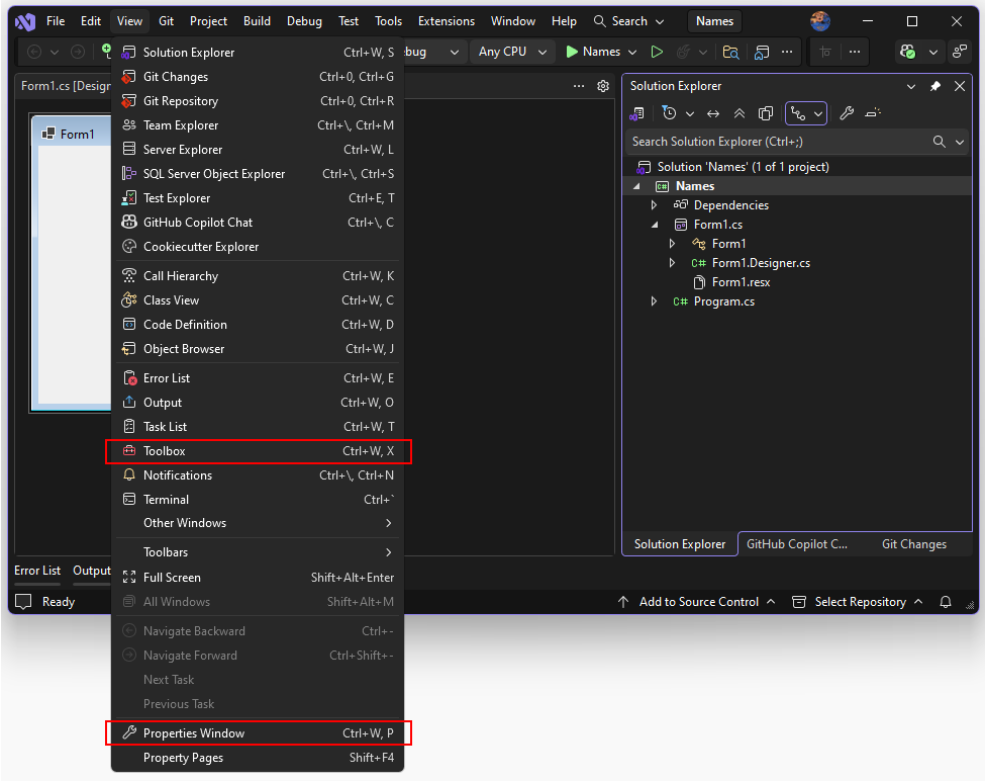


Рисунок 6 – Меню View с командами Toolbox и Properties Window

3.2. Найдите папку проекта

В Solution Explorer щёлкните узел проекта RaceLog правой кнопкой и выберите «Открыть папку в проводнике» (Open Folder in File Explorer). Сверьте путь с выбранным при создании. Для переноса работы сохраняйте всю папку решения. Не перемещайте по одному её внутренние элементы и не редактируйте их вне среды без необходимости.

В Solution Explorer узел Program относится к запуску приложения. Узел Form1 позволяет открыть форму и её код. Вложенная автоматически создаваемая часть формы хранит результат работы конструктора. Dependencies показывает зависимости. На первом занятии не переписывайте Program и автоматически создаваемую часть формы.

3.3. Проверьте пустое приложение

Выберите «Сборка», «Собрать решение» (Build, Build Solution). Если ошибок нет, нажмите F5 или выберите «Отладка», «Начать отладку» (Debug, Start Debugging). Должно появиться отдельное пустое окно Form1. Закройте

это окно перед следующим этапом. Если сборка не прошла, не соглашайтесь запускать старую успешную версию вместо текущей.

Что должно получиться

Вы знаете, где находится папка работы, умеете открыть конструктор и код формы, а пустое приложение уже запускается. Ошибки этой стадии нужно устранить до добавления собственного кода.

Этап 4. Создание формы для маленького примера

4.1. Настройте форму

Откройте конструктор Form1 и выберите саму форму: щёлкните её свободную область или заголовок. В Properties проверьте, что выбран объект Form1, а не будущая кнопка. Задайте Text: «Журнал заездов» и StartPosition: CenterScreen. Имя класса при этом остаётся Form1.

Для образца компоновки используйте ClientSize 980 на 500 и Font Segoe UI размером 10. Свойство ClientSize задаёт размер внутренней области окна. FormBorderStyle установите в FixedSingle, MaximizeBox в False. Это поможет новичку не отвлекаться на изменение размеров. Значения размеров выбраны для размещения элементов, а не для вычислений. Если при вашем масштабе Windows подписи не помещаются, допустимо увеличить окно.

4.2. Разместите четыре рабочих элемента

Из Toolbox перетащите на форму TextBox, Button, ListBox и Label. Для каждого выделенного элемента задайте (Name) в Properties. Это имя используется в коде. Text является видимым текстом, поэтому не путайте эти свойства. Расположите поле слева, кнопку под ним, список справа, а метку сообщения внизу.

Тип	**(Name)**	**Что задать**
TextBox	txtDriver	Text оставить пустым
Button	btnAdd	Text: Добавить
ListBox	lstResults	Items оставить пустым

Тип	(Name)	Что задать
Label	lblStatus	Text: Введите имя пилота

У lblStatus установите AutoSize в False и растяните метку так, чтобы она вмещала длинное сообщение. Добавьте ещё одну обычную Label над txtDriver с текстом «Пилот». Её можно назвать lblDriver. Эта подпись не участвует в обработчиках, но объясняет пользователю назначение поля.

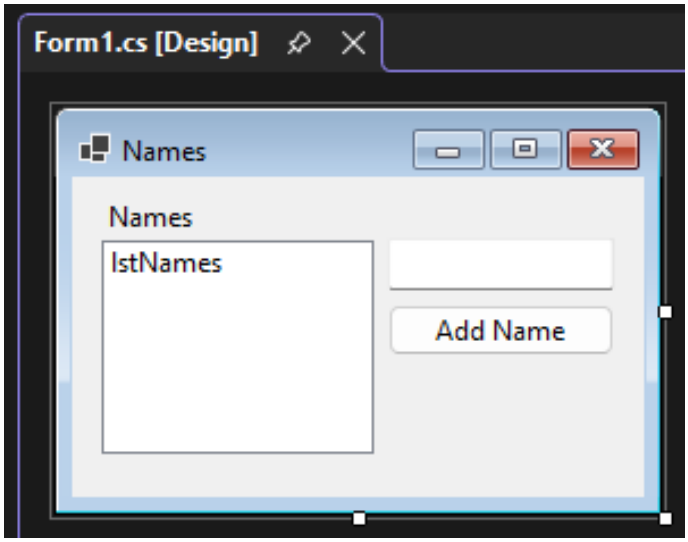


Рисунок 7 – Форма Names в конструкторе Windows Forms, это пример Names, он показывает приём размещения, а не готовый журнал заездов

Что должно получиться

Форма содержит поле имени, кнопку «Добавить», список и метку сообщения. Имена в Properties совпадают с таблицей. После F5 окно показывает эти элементы, но кнопка пока ничего не делает. Это ожидаемо: обработчик ещё не создан.

Этап 5. Пример: первая реакция кнопки

5.1. Создайте обработчик через конструктор

Завершите запуск приложения и вернитесь в конструктор. Выберите кнопку btnAdd. В Properties нажмите значок молнии, чтобы открыть события. Найдите строку Click и дважды щёлкните пустое поле справа. Visual Studio создаст метод btnAdd_Click, свяжет его с кнопкой и откроет редактор кода.

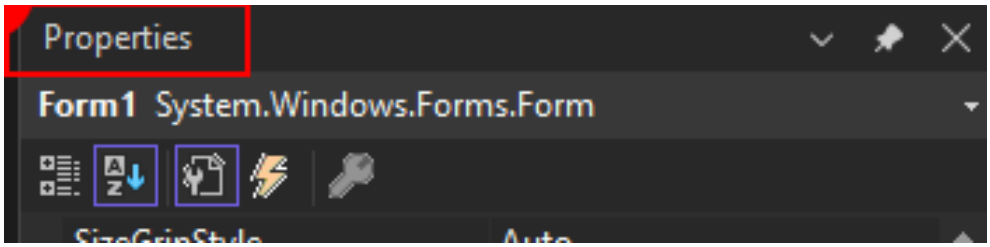


Рисунок 8 – Переключатель событий в окне Properties, фрагмент показывает положение значка молнии, здесь выбрана форма, а для создания обработчика сначала выделите btnAdd, после переключения найдите Click.

Убедитесь, что метод расположен внутри класса Form1. Код действия вводится между фигурными скобками этого метода. Не вставляйте ещё один класс Form1 и не помещайте второй обработчик внутрь уже созданного. Если Click уже заполнен, откройте существующий метод через это поле.

5.2. Прочитайте заготовку метода

private означает, что метод используется внутри класса формы. void означает отсутствие возвращаемого значения. sender сообщает, какой объект вызвал событие. e содержит данные события. object? допускает отсутствие объекта. В некоторых настройках среды параметр записан как object без вопросительного знака. Оставьте корректную заготовку, созданную средой, и не удаляйте параметры.

```
private void btnAdd_Click(object? sender, EventArgs e)
{
    lblStatus.Text = "Кнопка нажата";
}
```

Внутри метода добавлена одна команда. Слева от знака присваивания = находится свойство Text объекта lblStatus, справа новое строковое значение. Строка записывается в обычных двойных кавычках. Точка отделяет объект от его свойства, а точка с запятой завершает команду C#. Названия чувствительны к регистру: Text и text различаются.

Что должно получиться

После запуска остаётся исходное сообщение. Когда вы нажимаете «Добавить», оно меняется на «Кнопка нажата». Если надпись меняется сразу при запуске, команда попала не в обработчик. Если не меняется совсем, проверьте, что Click связан именно с btnAdd_Click.

Этап 5. Переменные, строки и чтение поля

5.3. Сохраните введённый текст в переменную

Переменная хранит значение под выбранным именем. Тип string предназначен для текста, int для целых чисел, bool для логических значений true и false. В объявлении сначала пишется тип, затем имя. Оператор = сохраняет значение правой части в указанном слева месте. Локальная переменная внутри обработчика создаётся при каждом его вызове.

```
string driver = txtDriver.Text;
lblStatus.Text = driver;
```

Замените прежнюю команду в btnAdd_Click этими двумя строками. Сначала берётся текст поля txtDriver и сохраняется в driver. Затем он выводится в метке. Введите «Анна», нажмите «Добавить» и проверьте сообщение. После изменения поля новый текст считывается при следующем нажатии, а не непрерывно.

5.4. Уберите пробелы по краям

Метод выполняет действие и вызывается с круглыми скобками. Метод Trim возвращает строку без пробелов в начале и конце. Внутренние пробелы между именем и фамилией остаются. Само поле txtDriver от этого вызова не очищается. Чтобы сохранить результат, измените первую строку.

```
string driver = txtDriver.Text.Trim();
```

Проверьте ввод с несколькими пробелами до и после имени. В driver останется «Анна». Это необходимо перед проверкой пустого имени: строка из одних пробелов после Trim становится пустой.

5.5. Различайте свойства и методы

Text и Length являются свойствами. Text содержит текст, Length сообщает длину строки. Clear является методом очистки поля, а Focus передаёт полю фокус клавиатуры. Вызовы выглядят как txtDriver.Clear() и txtDriver.Focus(). Их можно выполнить после успешного добавления, чтобы пользователь сразу вводил следующего пилота.

Что должно получиться

Вы умеете прочитать поле, сохранить строку в переменной и вывести её в метке. Список на этом шаге ещё не меняется. Не пытайтесь прибавлять или сравнивать текстовое значение времени как готовое число: это будет отдельно объяснено на этапе 7.

Этап 5. Условия и законченный пример

5.6. Проверьте пустое имя

Условие if выполняет блок команд только при истинном логическом выражении. Оператор == проверяет равенство, в отличие от присваивания =. Выражение driver.Length == 0 означает, что длина очищенной строки равна нулю. Фигурные скобки объединяют команды этого случая.

После сообщения об ошибке выполните return. Он заканчивает текущий вызов обработчика, но не закрывает приложение. Пользователь сможет исправить ввод и нажать кнопку снова. Без раннего выхода код после проверки продолжил бы выполняться.

5.7. Добавьте имя в список

У ListBox есть коллекция Items, то есть набор его элементов. Items.Add добавляет строку в конец, Items.Count показывает число строк. Отдельную коллекцию или счётчик пока создавать не нужно. Ниже приведён метод маленького примера целиком. Замените тело существующего обработчика, а не добавляйте второй метод с таким же именем.

```
private void btnAdd_Click(object? sender, EventArgs e)
{
    string driver = txtDriver.Text.Trim();
    if (driver.Length == 0)
    {
        lblStatus.Text = "Введите имя пилота";
        return;
    }
    lstResults.Items.Add(driver);
    txtDriver.Clear();
    txtDriver.Focus();
    lblStatus.Text = "Имя добавлено";
}
```

Что должно получиться

При пустом поле список не меняется. После ввода «Анна» появляется одна строка, поле очищается, фокус возвращается в него. При вводе имени с крайними пробелами в список попадает очищенная строка. Это завершённый пример, но ещё не решение основной задачи: автомобиль, трасса, время и удаление в нём отсутствуют.

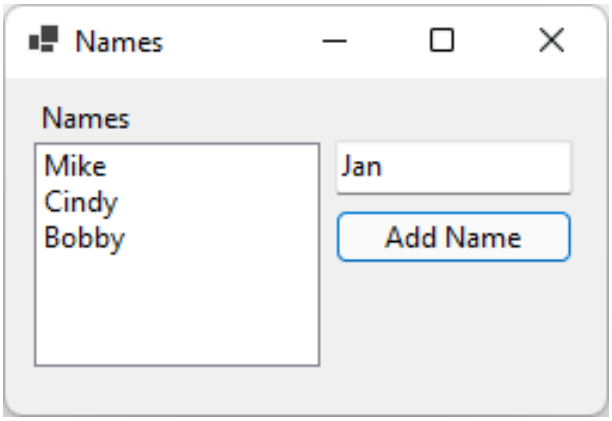


Рисунок 9 – Запущенное приложение Names, в нашей работе используется имя «Анна»

4. Самостоятельное задание

Продолжите собственный проект RaceLog, созданный на этапах 1-5. Получите журнал заездов с полями пилота и времени, выпадающими списками автомобиля и трассы, списком результатов и тремя кнопками. Самостоятельно напишите дополнительные обработчики и проверки ввода.

Какие данные использовать

Для автомобиля задайте три варианта: Street, Sport, Prototype. Для трассы задайте: Учебный овал, Городское кольцо, Спринт. Это учебные названия для выбора, а не характеристики реальных моделей. После запуска и очистки полей выбираются первые варианты обоих списков.

Для учебного тренировочного круга принимаются целые секунды от 30 до 600 включительно. Нижняя граница соответствует половине минуты, верхняя десяти минутам. Эти границы заданы условием для тренировки проверки ввода. Они не являются универсальным правилом автосимуляторов. Значения 30 и 600 принимаются, 29 и 601 отклоняются.

Что должно делать приложение

Кнопка «Добавить» читает имя, проверяет пустое имя, наличие выбора, целочисленность времени и диапазон. Только после успеха всех проверок она добавляет строку в формате «Имя | Автомобиль | Трасса | Время с». Например: «Анна | Street | Учебный овал | 94 с».

После добавления имя и время очищаются, выборы возвращаются к первым вариантам, а сообщение показывает текущее число результатов. Ошибочный ввод не меняет уже существующие результаты. Одинаковые строки разрешены: два тренировочных заезда могут иметь одинаковые данные.

Кнопка «Удалить выбранную» удаляет одну выбранную строку. При отсутствии выбора она выводит сообщение, не меняя список. Кнопка «Очистить поля» удаляет только незавершённый ввод и возвращает первые варианты списков. Результаты при этом остаются.

Не добавляйте в обязательную часть базу данных, сохранение между запусками, поиск, редактирование строк, асинхронность и сторонние библиотеки. Их отсутствие здесь не является недоработкой. После закрытия приложение начинает новый пустой журнал.

С чего продолжить

Сначала расширьте интерфейс на этапе 6. Затем на этапе 7 напишите логику добавления. На этапе 8 реализуйте удаление и очистку. Завершите этапом 9 и всеми 15 проверками раздела 7. Основная часть выполняется в том же проекте, а не в новом.

Этап 6. Элементы расширенной формы

6.1. Добавьте новые элементы

Завершите запуск. В конструкторе добавьте два ComboBox, ещё один TextBox, две Button и подписи Label. Сразу задайте имена, приведённые ниже. Уже существующие txtDriver, btnAdd, lstResults и lblStatus сохраняются. Обычные метки подписей можно назвать lblCar, lblTrack, lblTime и lblResults.

Элемент	(Name)	Подпись или назначение
ComboBox	cmbCar	Автомобиль
ComboBox	cmbTrack	Трасса
TextBox	txtTime	Время круга, с
Button	btnDelete	Удалить выбранную
Button	btnClear	Очистить поля

6.2. Заполните варианты ComboBox

Выберите cmbCar. В Properties установите DropDownStyle в DropDownList. Это запрещает ввод произвольного текста и оставляет выбор из списка. Найдите свойство Items, откройте редактор кнопкой справа от его значения и введите Street, Sport, Prototype, каждое значение на новой строке. Подтвердите ввод.

Так же заполните Items у cmbTrack названиями трёх трасс и задайте DropDownList. Не дублируйте эти строки дополнительными вызовами Items.Add в пользовательском коде, иначе варианты появятся дважды. Для списка результатов lstResults исходные Items должны оставаться пустыми.

6.3. Настройте удобство ввода

У txtDriver задайте TabIndex 0, у cmbCar 1, у cmbTrack 2, у txtTime 3, у btnAdd 4, у btnClear 5, у lstResults 6, у btnDelete 7. Это порядок перехода клавишей Tab. У lstResults включите HorizontalScrollbar, чтобы длинную запись можно было прочитать. Свойство SelectionMode оставьте One.

У формы задайте AcceptButton: btnAdd. Тогда Enter сможет запускать основное действие. Не используйте Enter вместо проверки кнопки мышью при первом подключении события. Сначала убедитесь, что сам обработчик работает.

Что должно получиться

На форме видны все поля, три кнопки, результаты и сообщение. Выпадающие списки содержат по три варианта и не позволяют вводить другие. Первоначальный выбор настроим следующим шагом.

Этап 6. Начальное состояние и место для кода

6.4. Откройте пользовательский код Form1

В Solution Explorer выберите Form1 и вызовите «Открыть код» (View Code). Найдите объявление класса и конструктор. public обозначает доступный класс или член, partial означает, что один класс хранится в нескольких частях. После двоеточия указан базовый класс Form. namespace RaceLog группирует программные имена нашего проекта.

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }
}
```

Этот фрагмент объясняет уже существующую структуру, а не предлагает вставить ещё один класс. Конструктор называется Form1, как и класс, и выполняется при создании окна. InitializeComponent создаёт и настраивает элементы по данным визуального конструктора. Удалять этот вызов нельзя.

6.5. Выберите первые варианты

В конструкторе после InitializeComponent добавьте две команды. Свойство SelectedIndex хранит индекс выбранного элемента. Первый элемент имеет индекс 0. Отрицательное значение -1 означает отсутствие выбора, а не первый вариант.

```
cmbCar.SelectedIndex = 0;
cmbTrack.SelectedIndex = 0;
```

Если элементы ещё не добавлены в Items, индекс 0 будет недопустим. Поэтому сначала выполняется визуальное заполнение, затем

InitializeComponent создаёт списки, затем команды выбирают первые значения. Не переносите эти команды выше InitializeComponent.

6.6. Различайте области вставки

Команды обработки нажатия пишутся внутри соответствующего метода. Новый вспомогательный метод пишется внутри класса Form1, но снаружи конструктора и других методов. Постоянные границы времени будут объявлены в классе перед конструктором. Автоматически создаваемую часть формы вручную не переписывайте.

Что должно получиться

После запуска выбраны Street и Учебный овал. Список результатов пуст. Уже проверенный маленький обработчик пока добавляет только имя. Его необходимо расширить следующим этапом.

Этап 7. Проверка имени, выбора и времени

7.1. Составьте порядок добавления

Откройте существующий btnAdd_Click через событие Click кнопки. Сохраните проверку имени из маленького примера. После неё должна идти проверка обоих ComboBox. Если хотя бы один SelectedIndex меньше нуля, покажите просьбу выбрать автомобиль и трассу и завершите обработчик. Символы || означают логическое ИЛИ: достаточно одного отсутствующего выбора.

7.2. Преобразуйте время

TextBox.Text возвращает строку даже тогда, когда в поле написано 94. Метод int.TryParse пытается получить целое число. Его возвращаемое значение имеет тип bool и показывает успех. Запись out int seconds одновременно объявляет переменную для выходного результата и передаёт её методу. После успешного преобразования seconds содержит число.

```
string timeText = txtTime.Text.Trim();
bool valid = int.TryParse(timeText, out int seconds);
```

Оператор ! означает отрицание. Условие !valid истинно, когда преобразование не удалось. В этом случае выведите сообщение «Время должно быть целым числом», верните фокус в txtTime и выполните return. Для слова «быстро», пустого поля и дробной записи корректное добавление не выполняется.

7.3. Введите постоянные границы

Внутри класса Form1 перед конструктором объявите две константы. const означает, что значение задано в объявлении и не меняется при работе программы. Имена обозначают нижнюю и верхнюю границы учебного тренировочного круга. Два символа // начинают комментарий до конца строки. Он объясняет код человеку и не выполняется программой. Рядом с границами оставьте комментарий об их смысле.

```
// Учебный круг: от половины минуты до десяти минут.
private const int MinLapSeconds = 30;
private const int MaxLapSeconds = 600;
```

После успешного преобразования проверьте, не меньше ли seconds нижней границы и не больше ли верхней. Для отказа эти причины соединяются через ||. Значения на самих границах должны пройти. Не используйте &&: оно потребовало бы одновременного нарушения двух границ, чего здесь не бывает.

Что должно получиться

Неверное имя, отсутствие выбора, неверный вид числа и выход за диапазон дают разные понятные сообщения. Пока не все проверки пройдены, список не должен изменяться. Код Items.Add расположите только после них.

Этап 7. Результат добавления и вспомогательные методы

7.4. Подготовьте видимую строку

Интерполяция вставляет значения в строку. Для этого перед открывающей кавычкой ставится $, а нужное выражение заключается внутри строки в фигурные скобки. Например, $"Время: {seconds} с" создаёт

сообщение с числом секунд. Символ | в требуемом виде записи нужен только для разделения частей.

После всех проверок самостоятельно соберите строку record из driver, cmbCar.Text, cmbTrack.Text и seconds. У ComboBox с DropDownList свойство Text содержит выбранный текст. Полученную строку добавьте через lstResults.Items.Add(record). Не добавляйте сначала одно имя, а затем второй полной строкой тот же результат.

7.5. Создайте ClearInput

Один и тот же набор очистки потребуется после добавления и по кнопке очистки. Создайте отдельный метод ClearInput внутри Form1 на одном уровне с btnAdd_Click. private ограничивает доступ классом формы, void означает отсутствие результата, пустые круглые скобки отсутствие параметров.

```
private void ClearInput()
{
    // Здесь самостоятельно напишите очистку полей.
}
```

Метод должен очистить txtDriver и txtTime вызовами Clear, выбрать индекс 0 в обоих ComboBox и вернуть фокус в txtDriver. Он не должен очищать lstResults и не должен задавать сообщение о конкретной операции. После добавления вызовите ClearInput(), затем покажите текст с текущим lstResults.Items.Count.

7.6. Создайте ShowError

Создайте метод private void ShowError(string message). Параметр message получает текст от вызывающего кода. Внутри присвойте его lblStatus.Text. Там, где обработчик обнаружил ошибку, вызовите ShowError с нужным текстом. После вызова по-прежнему нужен return в самом обработчике. Вспомогательный метод лишь показывает сообщение, а не прекращает вызывающий метод.

Что должно получиться

Корректный набор «Анна, Street, Учебный овал, 94» добавляется одной строкой. Поля очищаются, первые варианты выбираются снова, статус

показывает количество. После предыдущей ошибки успешная операция заменяет старое сообщение, а не оставляет его на экране.

Этап 8. Удаление и очистка текущего ввода

8.1. Создайте обработчик удаления

В конструкторе выберите btnDelete, откройте события Properties и создайте обработчик Click тем же способом, что для добавления. В btnDelete_Click получите int index = lstResults.SelectedIndex. Переменная index относится только к этому нажатию, хранить её между действиями в поле класса не требуется.

SelectedIndex равен -1, если ничего не выбрано. При index < 0 выведите «Сначала выберите запись в списке» и выполните return. Проверка Items.Count > 0 не заменяет эту проверку: в непустом списке тоже может не быть выбранной строки.

8.2. Удалите выбранную строку

Если выбор есть, вызовите Items.RemoveAt(index). Метод удаляет элемент по индексу. После удаления снова прочитайте Items.Count и обновите статус. Не удаляйте всегда элемент 0, иначе будет исчезать первая строка, а не выбранная. Не очищайте весь список ради удаления одной записи.

```
lstResults.Items.RemoveAt(index);
```

Проверьте три различные записи и удалите среднюю. Первая и последняя должны остаться. Индексы строк после удаления меняются автоматически. Поэтому для следующего нажатия снова считывается текущий SelectedIndex.

8.3. Создайте обработчик очистки

В Properties кнопки btnClear создайте обработчик её события Click. Внутри нового обработчика вызовите ClearInput, затем выведите сообщение «Поля очищены. Результаты сохранены в списке». Уточнение «в списке» означает сохранение текущих строк в памяти, а не запись на диск.

Перед проверкой добавьте результат, затем начните вводить второй. Нажмите «Очистить поля». Незавершённый ввод должен исчезнуть, но первая запись должна остаться. В этом обработчике не должно быть lstResults.Items.Clear(). Повторный метод ClearInput тоже не создавайте: вызывается тот, который уже объявлен на этапе 7.

Что должно получиться

Удаляется только выбранная запись. Удаление без выбора не завершает приложение аварийно. Очистка возвращает начальный ввод и не меняет результаты. Три кнопки связаны с тремя правильными обработчиками.

Этап 9. Сборка, отладка и поиск ошибок

9.1. Выполните текущую сборку

Сохраните изменения командой «Сохранить всё» (Save All). Выберите Build Solution. При ошибке откройте «Вид», «Список ошибок» (View, Error List). Дважды щёлкните первое осмысленное сообщение, чтобы перейти к месту. Одна незакрытая скобка может вызвать несколько последующих сообщений, поэтому не исправляйте список в случайном порядке.

Если имя элемента не найдено, сравните его написание в коде с (Name) в Properties и проверьте, что код находится в Form1. Если метод с таким именем уже существует, возможно, вы вставили второй обработчик вместо изменения первого. Если конструктор не открывается после удаления метода, проверьте оставшуюся связь события.

9.2. Посмотрите выполнение по шагам

Щёлкните в левом поле редактора рядом с первой командой btnAdd_Click. Появится точка останова. Нажмите F5, введите данные и нажмите «Добавить». Отладчик должен остановиться в обработчике. Наведите указатель на driver после выполнения его объявления и посмотрите значение. F10 выполняет следующую команду, F5 продолжает работу.

Подсвеченная текущая команда ещё не выполнена. Во время остановки отладчик приостанавливает программу, поэтому временный отказ окна

реагировать на ввод ожидаем. Чтобы увидеть значения без выхода из среды, можно открыть «Отладка», «Окна», «Локальные» (Debug, Windows, Locals).

9.3. Разберите частые ошибки поведения

Симптом	**Что проверить**
Кнопка не реагирует	Правильный метод в Click
Добавляются пробелы	Trim до проверки Length
29 или 601 проходят	Границы и оператор ||
Ошибка при удалении	SelectedIndex проверен до RemoveAt
Очистка уничтожает результаты	Нет очистки Items в ClearInput
Один клик добавляет дважды	Не дублируется связь события

Что должно получиться

Текущая сборка проходит, приложение запускается, вы можете остановиться в обработчике и объяснить значение переменной. После этого выполните все проверки следующего раздела. Сборка без ошибок сама по себе не доказывает правильность поведения.

5. Ожидаемый вид приложения

Схема показывает состав и расположение элементов. Точное совпадение размеров не требуется. Подписи и сообщения должны читаться, длинная строка должна быть доступна через прокрутку.

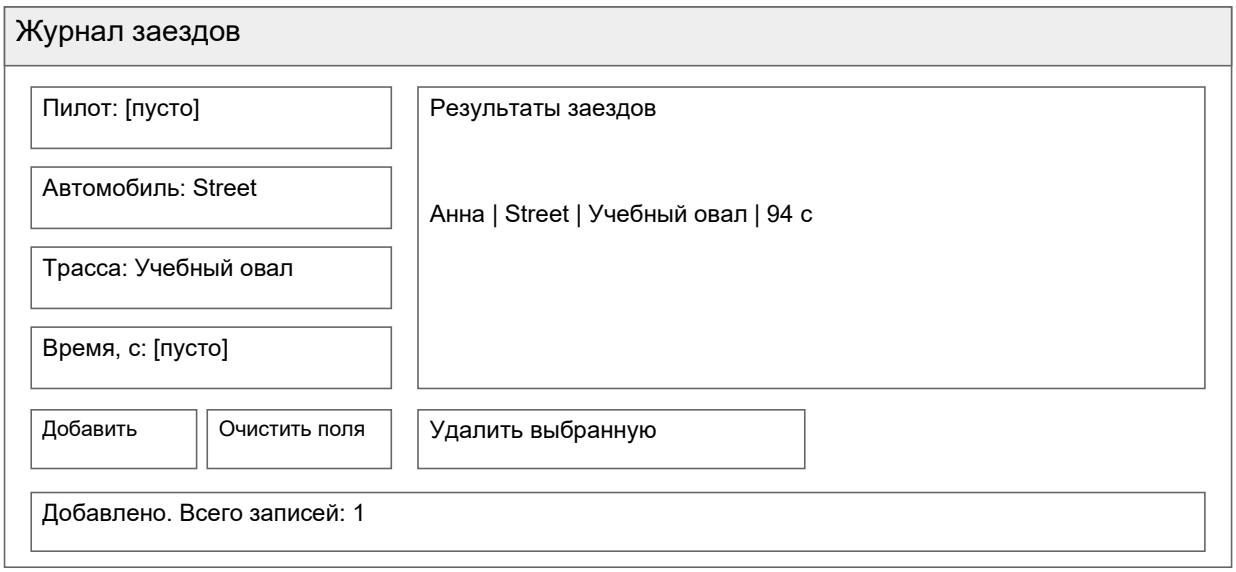


Рисунок 10 – Схема формы после добавления первой записи

Показано состояние после добавления. Поля очищены, выбраны первые варианты, запись остаётся в списке. При первом запуске список пуст. Ошибка выводится в той же метке без добавления строки.

Размеры для удобного повторения

Положение отсчитывается от левого верхнего угла клиентской области: сначала расстояние по горизонтали, затем по вертикали. Размеры выбраны для этой компоновки. При увеличенном масштабе Windows увеличьте форму, сохранив порядок элементов.

Объект	**Положение**	**Размер**
txtDriver	24, 54	250, 28
cmbCar	24, 124	250, 28
cmbTrack	24, 194	250, 28
txtTime	24, 264	250, 28
lstResults	304, 54	652, 296
lblStatus	24, 434	932, 48

6. Примеры данных и вид результата

Для проверки используйте одни и те же понятные данные. Они не являются внешним набором данных и не требуют загрузки. Это обычные учебные записи, которые нужно ввести в интерфейсе самостоятельно.

Пилот	**Автомобиль**	**Трасса**	**Секунды**
Анна	Street	Учебный овал	94
Иван	Sport	Городское кольцо	120
Мария	Prototype	Спринт	75

После ввода первой записи в ListBox должна появиться строка:

Анна | Street | Учебный овал | 94 с

После ввода всех трёх записей количество равно трём. Удалите вторую. Должны остаться результаты Анны и Марии, количество станет равным двум. Время каждой записи остаётся тем, которое было введено при её добавлении. Последующее изменение txtTime не должно менять существующие строки.

Различайте ошибку формата и ошибку диапазона

Значения «быстро», пустая строка, «94.5» и «94,5» не подходят для целого числа секунд. Значения 29 и 601 являются целыми числами, но не подходят по границам задания. Не выводите во всех случаях одинаковое сообщение «Ошибка»: пользователю нужно понимать, что исправить.

После успеха и после отказа

После успеха появляется ровно одна новая строка, очищаются поля и обновляется количество. После отказа список и количество остаются прежними. Неправильный ввод не надо стирать автоматически: пользователь должен иметь возможность исправить его. Для имени или времени верните фокус в соответствующее поле.

Объём самостоятельной реализации

В вашей форме должны находиться три обработчика Click и два вспомогательных метода ClearInput и ShowError. Кроме них, в конструкторе

задаётся первоначальный выбор, а в классе объявляются две границы времени. Полную логику расширенного добавления, удаления и очистки вы пишете сами. Не добавляйте неиспользуемые обработчики для всех доступных событий формы.

7. Проверка результата

Выполните 15 проверок. Для проверок времени используйте непустое имя и выбранные автомобиль и трассу. Иначе обработчик остановится на другой ошибке, и нужный случай не будет проверен. Перед ожидаемым отказом запомните количество строк и после действия убедитесь, что оно не изменилось.

Проверка 1. Первый запуск

Поля пусты, в списках выбраны Street и Учебный овал, результатов нет.

Проверка 2. Пустое имя и время 90

Запись не добавляется. Появляется просьба ввести имя, фокус в txtDriver.

Проверка 3. Имя из пробелов и время 90

После Trim имя считается пустым. Список не меняется.

Проверка 4. Имя Анна, время пустое или «быстро»

Оба ввода отклоняются с сообщением о целом числе.

Проверка 5. Имя Анна, время 94.5 и затем 94,5

Обе дробные записи отклоняются, поскольку нужны целые секунды.

Проверка 6. Имя Анна, время 29

Запись не добавляется. Сообщение указывает допустимый диапазон.

Проверка 7. Имя Анна, время 30

Нижняя граница принимается. Число результатов увеличивается на один.

Проверка 8. Имя Анна, время 94

В списке появляется «Анна | Street | Учебный овал | 94 с».

7. Проверка результата (продолжение)

Проверка 9. Имя Анна, время 600

Верхняя граница принимается.

Проверка 10. Имя Анна, время 601

Значение выше верхней границы отклоняется.

Проверка 11. Имя с пробелами по краям, время 94

В результате имя без крайних пробелов. После успеха поля имени и времени пусты.

Проверка 12. Удаление без выбранной строки

Появляется просьба выбрать запись. Число результатов не меняется.

Проверка 13. Удаление выбранной средней строки из трёх

Удаляется только эта строка. Остальные сохраняются, количество уменьшается на один.

Проверка 14. Очистка полей при непустом списке

Поля очищаются, выборы возвращаются к первым вариантам, результаты остаются.

Проверка 15. Закрытие и повторный запуск

Список снова пуст. Сохранение между запусками в ЛР 1 не предусмотрено.

В отдельной проверке удобства пройдите поля клавишей Tab, попробуйте Enter для добавления и проверьте длинную строку с прокруткой. Если какая-то проверка не проходит, исправьте код и повторите связанные сценарии. Например, после изменения ClearInput снова проверьте добавление и отдельную очистку.

8. Что представить в отчёте и на защите

Краткий отчёт

Укажите название разработанного приложения и используемые Visual Studio и .NET. Коротко опишите, что оно делает. Покажите снимки своего

интерфейса: начальное состояние, корректное добавление и одно сообщение об ошибке. Подпишите, какие данные вводились и что произошло.

Приведите собственный обработчик добавления и объясните порядок его проверок. Отдельно поясните, зачем нужны ClearInput и ShowError. Не перепечатывайте всю теорию и автоматически создаваемый код размещения элементов. По каждой из 15 проверок укажите наблюдаемый результат. Удобно отметить номер сценария, введённые значения и краткий вывод.

Опишите одну ошибку, которую вы действительно обнаружили, и её исправление. Если не встретили ошибок, не придумывайте их: вместо этого разберите один потенциальный неверный вариант, например удаление без проверки индекса. Укажите ограничение первой версии: результаты существуют только до закрытия приложения.

Демонстрация приложения

Откройте собственный проект в Visual Studio, выполните сборку и запустите приложение. Покажите добавление результата, отказ при неверном имени и времени, принятие 30 и 600, отклонение 29 и 601, удаление выбранного элемента и очистку незавершённого ввода без потери результатов.

В конструкторе покажите имена элементов и связь Click с соответствующим методом. В редакторе покажите конструктор Form1, три обработчика и два вспомогательных метода. Объясните, почему Items.Add находится после проверок и что делает return при ошибке. По просьбе преподавателя поставьте точку останова и покажите значение переменной.

Что передать преподавателю

Передайте свою работу целиком так, чтобы её можно было открыть как решение и собрать на другом учебном компьютере, а также краткий отчёт. Способ передачи определяет преподаватель. Отдельные снимки или скопированный текст одного метода не заменяют проект.

Когда работа считается выполненной

Созданное вами приложение запускается, выполняет все обязательные действия и проходит проверки. Вы можете объяснить использованные элементы и код. Внешнее сходство с образцом без работающей логики или чужое готовое решение не подтверждают выполнение работы.

9. Контрольные вопросы

1. Какие действия в работе показывают прототипирование и итеративность?

2. Чем Visual Studio отличается от C#, .NET и Windows Forms?

3. Чем проект отличается от решения и где находится папка вашей работы?

4. Для чего нужны Toolbox, Properties, Form Designer и Solution Explorer?

5. Чем (Name) элемента отличается от его свойства Text?

6. Что такое класс Form1 и когда выполняется его конструктор?

7. Зачем нужен InitializeComponent и почему его нельзя удалять?

8. Чем событие Click отличается от обработчика btnAdd_Click?

9. Как создать обработчик и проверить его связь с кнопкой?

10. Что означают string, int и bool? Чем = отличается от ==?

11. Почему TextBox.Text нужно преобразовывать перед числовым сравнением?

12. Что делает Trim и меняет ли он сам текст поля?

13. Какие результаты возвращает int.TryParse и что означает out?

14. Почему после ошибки нужен return и завершает ли он приложение?

15. Почему проверка выхода за границы использует ||, а не &&?

16. Что означает SelectedIndex = -1 и чем индекс отличается от Count?

17. Чем отличаются объявление ClearInput и его вызов?

18. Почему очистка текущего ввода не должна очищать список результатов?

19. Как с помощью точки останова посмотреть значение driver?

20. Почему после повторного запуска результаты исчезают?